in sampling (§5.2); complaints name the reference they concern (§5.3); and selection counts hard failures (e.g., a missing reference) before pairwise judgment, and a later draft replaces the incumbent only if it wins under this rule (§5.4). Each component was added in an iteration that raised the validation score, and alternatives like editing the winner in place were dropped (Appendix D).

5.1 Reference-Grounded Prompting

References play different roles (identity, garment, attribute, background, style); a generator that confuses them leaks attributes or drops references. AutoRef-Harness never passes the raw instruction to the generator: M_θ reads the instruction and all references and writes a prompt that assigns each requested element to its reference, excluding unrequested content; later prompts use the same format.

5.2 Structurally Diverse Drafts

A common failure is a pasted-in look: each subject matches its reference, but its lighting, perspective, or colors disagree with the scene. Resampling one prompt rarely fixes this, so the two drafts use different prompt structures. Draft A describes the scene subject by subject. For draft B, M_θ identifies the reference designated as the background or style, and the prompt asks the generator to keep that reference as the canvas and paint the other subjects into it, so that subjects and scene are rendered jointly. If no such reference exists, draft B is a second sample of draft A's prompt.

5.3 Complaint-Directed Revision

M_θ lists up to five concrete complaints about the winner of A and B, each naming the reference it concerns (e.g., wrong identity, attribute from the wrong reference, inconsistent lighting), and rewrites the prompt to address them; draft C is generated from the revised prompt, or by resampling the winner's prompt if there is no complaint.

5.4 Failure-Aware Selection

Candidates are compared in pairs. Each draft is first checked for hard failures (missing reference, extra or duplicated subject, wrong background); the draft with fewer failures wins. On a tie, M_θ lists the differences a strict rater would score and names a winner in both presentation orders; the challenger (draft B, then draft C) must win both. Selection uses M_θ (GPT-5.5), not the evaluator R.

6 Experiments

6.1 Experimental Settings

Benchmarks. We evaluate on MultiBanana (Oshima et al., 2026), a benchmark for multi-reference image generation. We use the 229 tasks with four reference images, split into 48 training tasks, 48 validation tasks, and 133 test tasks, with Qwen3-VL-8B-Instruct (Bai et al., 2025) as the evaluator. We use the training and validation splits for harness optimization, while the held-out test split remains unseen during search. To test generalization across unseen reference counts, we further evaluate on the three- and five-reference settings, randomly sampling 24 tasks per task type (96 per setting).

To evaluate generalization beyond the benchmark and evaluator, we also test on OmniContext (Wu et al., 2026a), which we never use during harness search. We randomly sample 15 tasks from each task type, for 120 tasks in total, and evaluate them using the official GPT-4.1 (OpenAI, 2023) evaluator. With both the benchmark and the evaluator differing from those used in the search, this setting tests whether the learned harness transfers to unseen data distributions and evaluation signals.

Harness Search. We initialize the search with two harnesses: the base FLUX.2 [klein] 4B generator (Black Forest Labs, 2026) and GEMS (He et al., 2026b), an image-generation harness configured with FLUX.2 [klein] 4B as the generator and GPT-5.5 (OpenAI, 2026) as the reasoning model. For AutoRef (Section 4), we use Claude Fable 5.1 as the proposer through the Claude Code CLI (Anthropic, 2025) and run five search iterations. Implementation details and model versions are in Appendix A, and the proposer's prompts are in Appendix E.